

Translated

From: e-maxx.ru

Balanced bracket sequences

A **balanced bracket sequence** is a string consisting of only brackets, such that this sequence, when inserted certain numbers and mathematical operations, gives a valid mathematical expression. Formally you can define balanced bracket sequence with:

- ϵ (the empty string) is a balanced bracket sequence.
- if s is a balanced bracket sequence, then so is (s) .
- if s and t are balanced bracket sequences, then so is $s t$.

For instance $((()))$ is a balanced bracket sequence, but $((()())$ is not.

Of course you can define other bracket sequences also with multiple bracket types in a similar fashion.

In this article we discuss some classic problems involving balanced bracket sequences (for simplicity we will only call them sequences): validation, number of sequences, finding the lexicographical next sequence, generating all sequences of a certain size, finding the index of sequence, and generating the k -th sequences. We will also discuss two variations for the problems, the simpler version when only one type of brackets is allowed, and the harder case when there are multiple types.

Balance validation

We want to check if a given string is balanced or not.

At first suppose there is only one type of bracket. For this case there exists a very simple algorithm. Let depth be the current number of open brackets. Initially $\text{depth} = 0$. We iterate over all character of the string, if the current bracket character is an opening bracket, then we increment depth , otherwise we decrement it. If at any time the variable depth gets negative, or at the end it is different from 0 , then the string is not a balanced sequence. Otherwise it is.

If there are several bracket types involved, then the algorithm needs to be changed. Instead of a counter depth we create a stack, in which we will store all opening brackets that we meet. If the current bracket character is an opening one, we put it onto the stack. If it is a